

Computer Organization

From Digital Logic to Computer Systems

Dr. Neeraj Sharma

School of Computing and Electrical Engineering
Indian Institute of Technology Mandi

Why Study Computer Organization?

A Question for Computer Science Students

Two algorithms may have similar theoretical complexity, but do they necessarily have the same performance on a real computer?

Algorithmic View

We normally study:

- Time complexity
- Space complexity
- Asymptotic analysis
- $O(n)$, $O(n \log n)$, $O(n^2)$

Computer Organization View

Actual execution also depends on:

- Cache behavior
- Memory accesses
- CPU architecture
- Branch prediction
- Data locality

Key Idea

**Complexity tells us how an algorithm scales.
Computer organization helps explain how fast it actually runs.**

Motivating Example: Merge Sort vs Quick Sort

Consider two commonly used sorting algorithms.

Merge Sort

$$T(n) = O(n \log n)$$

- Divide array into two halves
- Recursively sort both halves
- Merge the sorted arrays
- Usually requires auxiliary memory

Quick Sort

Average case:

$$T(n) = O(n \log n)$$

Worst case:

$$T(n) = O(n^2)$$

- Select a pivot
- Partition the array
- Recursively sort partitions
- Mostly operates in-place

Question

If Merge Sort has guaranteed $O(n \log n)$ complexity while Quick Sort can become $O(n^2)$, why is Quick Sort often very fast for arrays?

Same Complexity $\neq$ Same Execution Time

For sufficiently large input:

$$T_{\text{merge}}(n) \approx c_m n \log n$$

and

$$T_{\text{quick}}(n) \approx c_q n \log n$$

The constants c_m and c_q are influenced by the hardware.

Quick Sort

- Mostly sequential array access
- Good spatial locality
- In-place partitioning
- Small auxiliary memory
- Good cache utilization

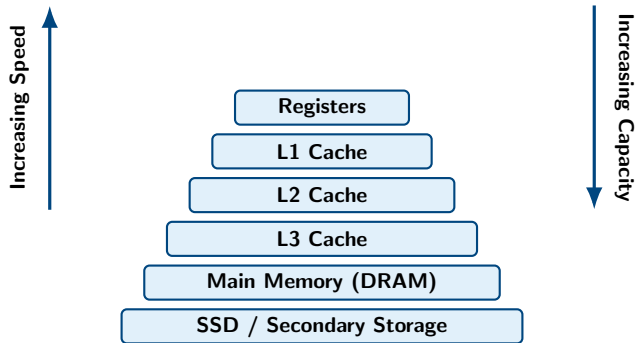
Merge Sort

- Reads from source arrays
- Writes into temporary arrays
- More memory traffic
- Requires auxiliary storage
- Performance can become memory-bound

Important

Big- O notation does not model cache misses, memory latency, instruction cost, or processor implementation.

Memory Hierarchy: Why Cache Matters



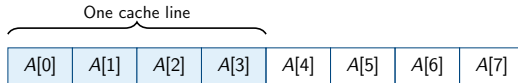
Connection to Sorting

Quick Sort's in-place partitioning often provides good **spatial locality**, contributing to its strong practical performance on arrays.

Cache Lines and Spatial Locality

How Data Moves Between Memory and Cache

The processor generally does not fetch a single byte from main memory. Instead, it fetches a complete **cache line**.



Sequential Access

$A[0], A[1], A[2], A[3], \dots$

- Nearby elements reused
- Fewer cache misses
- Good spatial locality

Random Access

$A[0], A[1000], A[5], A[9000], \dots$

- Data scattered in memory
- More cache misses
- Higher memory latency

Takeaway

Two algorithms with identical Big- O complexity can behave very differently depending on their memory access patterns.

What Do Programming Libraries Actually Use?

Platform	Typical Algorithm	Reason
C++ <code>std::sort</code>	Introsort	Fast average case + worst-case protection
Python <code>sort()</code>	TimSort	Stable and excellent on partially sorted data

Example: Introsort

Introsort combines:

Quick Sort + Heap Sort + Insertion Sort

- Start with Quick Sort
- Switch to Heap Sort if recursion becomes too deep
- Use Insertion Sort for very small partitions

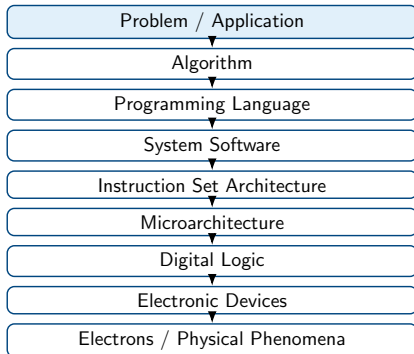
Engineering Lesson

The “best” algorithm often depends on both **theoretical guarantees** and **hardware behavior**.

From a Problem to Running Hardware

A Computer System is a Hierarchy of Abstractions

A user starts with a real-world problem, while the physical machine ultimately manipulates electrical signals.



Computer Organization

Computer Organization mainly explains how the middle and lower layers work together to execute software efficiently.

Layer 1: Problem and Algorithm

Problem

A real-world requirement is first converted into a computational problem.

Examples:

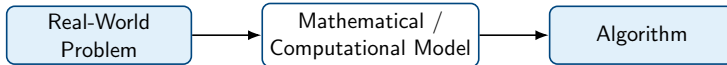
- Sort student records
- Search a database
- Recognize an image
- Route packets
- Multiply matrices

Algorithm

An algorithm describes the logical steps required to solve the problem.

Examples:

- Quick Sort
- Binary Search
- Dijkstra's Algorithm
- Matrix multiplication
- Neural-network inference



Important

An algorithm is independent of the programming language and processor, but its practical performance can depend strongly on the hardware.

Layer 2: Programming Language and System Software

Programming Language

Algorithms are expressed using languages such as:

- C / C++
- Java
- Python
- Rust

Example:

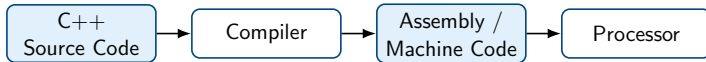
```
int c = a + b;
```

The processor does not directly understand this statement.

System Software

System software translates and manages program execution.

- Compiler
- Assembler
- Linker
- Loader
- Operating System
- Runtime libraries



Next Question

What machine instructions does the compiler generate, and how does the processor understand them?

Layer 3: Instruction Set Architecture (ISA)

What is an ISA?

The **Instruction Set Architecture** is the interface between software and hardware.

ISA Defines

- Machine instructions
- Registers
- Data types
- Addressing modes
- Memory access rules
- Instruction encoding

Examples

- x86 / x86-64
- ARM
- RISC-V
- MIPS



Key Point

Different processors can implement the same ISA using different microarchitectures.

x86 vs ARM vs RISC-V

Feature	x86	ARM	RISC-V
Design style	Traditionally complex instruction set	Reduced instruction set	Open reduced instruction set
Typical use	Desktop, laptop, server	Mobile, embedded, increasingly servers	Research, embedded, custom processors
Instruction encoding	Variable length	Mostly fixed / structured	Modular and extensible
Examples	Intel Core, AMD Ryzen	Apple Silicon, Cortex series	SiFive, many research processors

Important

The ISA specifies **what** the processor should do. It does not specify exactly **how** the processor internally does it.

Example

Two processors may both execute x86-64 instructions, but differ in cache size, pipeline depth, branch prediction, execution units and performance.

Layer 4: Microarchitecture

What is Microarchitecture?

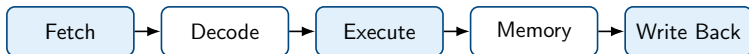
Microarchitecture is the internal organization used to implement an ISA.

Major Components

- Registers
- ALU
- Cache
- Control unit
- Pipeline
- Execution units

Performance Features

- Branch prediction
- Superscalar execution
- Out-of-order execution
- Instruction-level parallelism
- Prefetching



ISA vs Microarchitecture

ISA = programmer-visible specification

Microarchitecture = internal hardware implementation

Layer 5: Digital Logic

Microarchitecture is Built from Digital Logic

At the next abstraction level, processors are implemented using logic gates and storage elements.

Combinational Circuits

Output depends only on current inputs.

Examples:

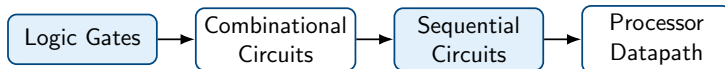
- AND, OR, NOT gates
- Multiplexers
- Encoders / Decoders
- Adders
- ALU

Sequential Circuits

Output depends on current input and previous state.

Examples:

- Latches
- Flip-flops
- Registers
- Counters
- State machines



Layer 6: Electronic Devices

Logic Gates Are Built from Electronic Devices

Digital logic is physically implemented using semiconductor devices.

Diode

Allows current mainly in one direction.

Used in:

- Protection
- Rectification
- Early logic circuits

BJT

Current-controlled transistor.

Used historically in:

- TTL logic
- Amplifiers

MOSFET

Voltage-controlled transistor.

Dominant device in:

- CPUs
- Memory
- Digital ICs

Modern Digital Systems

Modern processors contain billions of MOSFETs integrated on a single semiconductor chip.

MOSFET as a Digital Switch

Basic Idea

A MOSFET can be treated approximately as an electronically controlled switch in digital logic.

nMOS

$V_G = 1 \Rightarrow \text{ON}$

$V_G = 0 \Rightarrow \text{OFF}$

nMOS is good at pulling an output toward logic 0.

pMOS

$V_G = 0 \Rightarrow \text{ON}$

$V_G = 1 \Rightarrow \text{OFF}$

pMOS is good at pulling an output toward logic 1.



Why Both nMOS and pMOS?

CMOS logic combines nMOS and pMOS devices so that outputs can be driven strongly to both logic 0 and logic 1 while keeping static power consumption very low.

Layer 7: Electrons and Physical Representation

What is Actually Happening at the Lowest Level?

At the physical level, computation is ultimately represented through **electrical quantities** and the controlled movement of charge.

Physical Quantities

- Voltage
- Current
- Electric charge
- Electric field
- Capacitance

Digital Abstraction

Instead of reasoning about individual electrons, we define voltage ranges as:

LOW $\Rightarrow$ 0

HIGH $\Rightarrow$ 1



Abstraction is Powerful

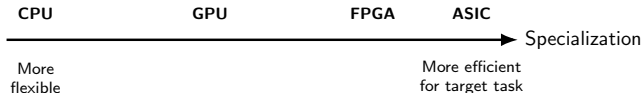
A programmer does not need to track electrons while executing `a+b`; each layer hides unnecessary lower-level details.

General-Purpose vs Specialized Computing

One Architecture is Not Best for Every Problem

Different applications require different combinations of **performance, flexibility, parallelism, power and cost.**

	CPU	GPU	FPGA	ASIC
Purpose	General purpose	Highly parallel computation	Reconfigurable hardware	Application-specific
Flexibility	Very high	High	High after re-design	Very low
Parallelism	Moderate	Very high	Custom	Custom / very high
Efficiency	Moderate	High for parallel workloads	High	Highest for target task
Typical use	OS, applications	Graphics, AI, scientific computing	Prototyping, networking, acceleration	AI chips, mining, codecs



Why Build Application-Specific Hardware?

Example 1: Artificial Intelligence

Neural networks repeatedly perform operations such as:

$$C = A \times B$$

Matrix multiplication contains a very large number of independent multiply-accumulate operations. Therefore, hardware can provide many parallel execution units.

General-Purpose CPU

- Designed for many workloads
- Complex control logic
- Limited number of powerful cores
- Maximum flexibility

AI Accelerator

- Large number of MAC units
- Massive parallelism
- Optimized data movement
- Better performance

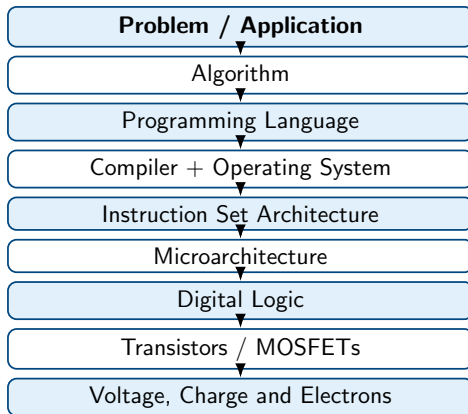
Example 2: Bitcoin Mining

Bitcoin mining repeatedly evaluates SHA-256 hashing operations. Specialized ASICs can implement the required operations directly in hardware.

Hardware-Software Co-Design

If an application is sufficiently important and computationally expensive, we may design hardware specifically for that workload.

Putting Everything Together



The Central Idea of Computer Organization

Software performance cannot be fully understood without understanding the hardware on which the software executes.

Where Do We Start?

Digital logic understands two states:



To understand how computers represent and manipulate data using **0s and 1s**, we first need to understand

NUMBER SYSTEMS

- A number system is a systematic way to represent and manipulate numbers.
- Examples: decimal, binary, octal, hexadecimal, Roman, sexagesimal, Gray code.
- Broad classification:
 - **Weighted**: digit positions have weights, e.g., decimal, binary, octal, hexadecimal.
 - **Non-weighted**: no simple positional weight, e.g., Roman numerals, Gray code.

Decimal number system

- Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Base/radix: 10.
- Every digit position has a weight that is a power of 10.

$$234 = 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0$$

$$250.67 = 2 \times 10^2 + 5 \times 10^1 + 0 \times 10^0 + 6 \times 10^{-1} + 7 \times 10^{-2}$$

Binary number system

- Digits: 0 and 1.
- Base/radix: 2.
- Each position has weight 2^i .
- Binary digits are called bits.

$$1102 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 610$$

$$101.012 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 5.2510$$

General radix-based number system

For a radix r , allowed digits are normally $0, 1, 2, \dots, r - 1$.

$$N = (a_{n-1}a_{n-2} \dots a_1a_0.a_{-1}a_{-2} \dots a_{-m})_r$$

$$N = \sum_{i=-m}^{n-1} a_i r^i$$

Key idea

The same digit has different value depending on its position.

Binary to decimal conversion

Rule

Multiply every bit by its positional weight and add.

$$B = b_{n-1}b_{n-2} \dots b_1b_0.b_{-1}b_{-2} \dots b_{-m}$$

$$D = \sum_i b_i 2^i$$

Example

$$1010112 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 = 4310$$

More binary to decimal examples

$$\begin{aligned}.01012 &= 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} \\ &= 0.25 + 0.0625 = .3125_{10}\end{aligned}$$

$$\begin{aligned}101.112 &= 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ &= 5.75_{10}\end{aligned}$$

Decimal integer to binary

Repeated division method

Repeatedly divide the decimal integer by 2 and collect remainders. The binary number is obtained by reading the remainders upward.

Example: $(239)_{10}$

$239 \rightarrow 119 \text{ } r1 \rightarrow 59 \text{ } r1 \rightarrow 29 \text{ } r1 \rightarrow 14 \text{ } r1 \rightarrow 7 \text{ } r0 \rightarrow 3 \text{ } r1 \rightarrow 1 \text{ } r1 \rightarrow 0 \text{ } r1$

Reading remainders upward:

$$239_{10} = 11101111_2$$

Decimal fraction to binary

Repeated multiplication method

Repeatedly multiply the fraction by 2. The integer parts generated form the binary fraction from left to right.

Example: $(0.625)_{10}$

$$0.625 \times 2 = 1.25 \Rightarrow 1$$

$$0.25 \times 2 = 0.5 \Rightarrow 0$$

$$0.5 \times 2 = 1.0 \Rightarrow 1$$

$$0.625_{10} = 0.101_2$$

Why octal and hexadecimal?

- Binary is natural for hardware, but long binary strings are difficult for humans.
- Octal groups binary bits in groups of 3.
- Hexadecimal groups binary bits in groups of 4.
- Hexadecimal is widely used for addresses, machine code, masks, colors, and memory dumps.

Computer organization connection

One hexadecimal digit exactly represents one nibble, so a 32-bit word requires only 8 hex digits.

Octal number system

- Base/radix: 8.
- Digits: 0 to 7.
- One octal digit represents 3 binary bits.

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Binary to octal conversion

- Integer part: group bits from right to left in groups of three.
- Fractional part: group bits from left to right in groups of three.
- Add leading or trailing zeros if required.

Examples

$$101\ 101\ 000\ 011_2 = 5503_8$$

$$1\ 010\ 100\ 001_2 = 1241_8$$

$$.100\ 001\ 100_2 = .414_8$$

Octal to binary conversion

Rule

Replace every octal digit by its 3-bit binary equivalent.

$$16458 = 001\ 110\ 100\ 101_2$$

$$22.1728 = 010\ 010.001\ 111\ 010_2$$

$$1.548 = 001.101\ 100_2$$

Leading zeros may be removed from the integer part.

Hexadecimal number system

- Base/radix: 16.
- Digits: 0 to 9, *A* to *F*.
- One hex digit represents 4 binary bits.

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Binary to hexadecimal conversion

- Integer part: group bits from right to left in groups of four.
- Fractional part: group bits from left to right in groups of four.
- Add leading or trailing zeros if required.

Examples

$1011\ 0100\ 0011_2 = B43_{16}$

$10\ 1010\ 0001_2 = 2A1_{16}$

$.1000\ 0100_2 = .84_{16}$

$101.0101\ 1110_2 = 5.5E_{16}$

Hexadecimal to binary conversion

Rule

Replace every hexadecimal digit by its 4-bit binary equivalent.

$$3A5_{16} = 0011\ 1010\ 0101_2$$

$$12.3D_{16} = 0001\ 0010.0011\ 1101_2$$

$$1.8_{16} = 0001.1000_2$$

Decimal to radix- r and radix- r to decimal

- Radix- r to decimal: multiply each digit by its positional weight and add.
- Decimal integer to radix- r : repeated division by r .
- Decimal fraction to radix- r : repeated multiplication by r .

Example

$$2A116 = 2 \times 16^2 + 10 \times 16^1 + 1 \times 16^0 = 67310$$

How many values can n bits represent?

Each bit has two possible states.

$$\underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ terms}} = 2^n$$

Important distinction

The same 2^n bit patterns may be interpreted differently depending on the representation scheme.

- Unsigned integer: only magnitude.
- Signed integer: magnitude plus positive/negative meaning.

Unsigned binary numbers

An n -bit unsigned integer ranges from:

$$0 \text{ to } 2^n - 1$$

For bits $b_{n-1} \dots b_1 b_0$:

$$D = b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \dots + b_12^1 + b_02^0$$

For $n = 3$

000, 001, 010, 011, 100, 101, 110, 111 $\Rightarrow$ 0 to 7

Unsigned representation in 4 bits

Decimal	Binary	Decimal	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Signed integer representation

Many numerical data items are signed.

Question

How should sign be represented using only bits?

Three common approaches:

- ① Sign-magnitude representation
- ② One's complement representation
- ③ Two's complement representation

Modern processors primarily use two's complement for signed integers.

Sign-magnitude representation

- MSB represents sign: 0 for positive, 1 for negative.
- Remaining $n - 1$ bits represent magnitude.

$$\text{Range} = -(2^{n-1} - 1) \text{ to } +(2^{n-1} - 1)$$

Limitation

There are two zeros: $+0 = 000 \dots 0$ and $-0 = 100 \dots 0$.

Sign-magnitude representation in 4 bits

Decimal	Sign-mag	Decimal	Sign-mag
+0	0000	-0	1000
+1	0001	-1	1001
+2	0010	-2	1010
+3	0011	-3	1011
+4	0100	-4	1100
+5	0101	-5	1101
+6	0110	-6	1110
+7	0111	-7	1111

One's complement representation

- Positive numbers are represented as ordinary binary.
- Negative numbers are represented by complementing every bit of the positive magnitude.

Example in 4 bits

$$+5 = 0101, \quad -5 = 1010$$

$$\text{Range} = -(2^{n-1} - 1) \text{ to } +(2^{n-1} - 1)$$

Limitation

Again, there are two zeros: $000 \dots 0$ and $111 \dots 1$.

One's complement representation in 4 bits

Decimal	1's comp.	Decimal	1's comp.
+0	0000	-7	1000
+1	0001	-6	1001
+2	0010	-5	1010
+3	0011	-4	1011
+4	0100	-3	1100
+5	0101	-2	1101
+6	0110	-1	1110
+7	0111	-0	1111

Two's complement representation

- Positive numbers are represented as ordinary binary.
- To obtain $-x$, complement every bit of x and add 1.

Example in 4 bits

$$+5 = 0101$$

Complement: 1010, Add 1: 1011

$$-5 = 1011$$

Two's complement range

For n bits:

$$\text{Minimum} = -2^{n-1}, \quad \text{Maximum} = 2^{n-1} - 1$$

Advantages

- Unique representation of zero.
- Subtraction can be performed using addition.
- Hardware implementation is simpler than sign-magnitude arithmetic.

Two's complement representation in 4 bits

Decimal	2's comp.	Decimal	2's comp.
+0	0000	-8	1000
+1	0001	-7	1001
+2	0010	-6	1010
+3	0011	-5	1011
+4	0100	-4	1100
+5	0101	-3	1101
+6	0110	-2	1110
+7	0111	-1	1111

Weighted interpretation of two's complement

For bits $b_{n-1}b_{n-2} \dots b_1b_0$:

$$D = -b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \dots + b_12^1 + b_02^0$$

Example

$$1101 = -1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = -8 + 4 + 1 = -3$$

Sign extension

Rule

To increase the number of bits in a two's complement number, copy the sign bit to the left.

+22 : 00010110 → 0000000000010110

−10 : 11110110 → 1111111111110110

This preserves the numerical value.

Shifts in two's complement

- Left shift by k positions with zero padding usually multiplies by 2^k if no overflow occurs.
- Arithmetic right shift by k positions with sign-bit padding divides by 2^k approximately, preserving sign.

Example

$$00010110 = +22$$

$$\text{Arithmetic right shift by 2: } 00000101 = +5$$

Subtraction using one's complement

To compute $A - B$:

- ① Compute one's complement of B : call it B_1 .
- ② Add $A + B_1$.
- ③ If a carry is produced, add it back to the result. This is called end-around carry.
- ④ If no carry is produced, the result is negative and in one's complement form.

One's complement subtraction: $6 - 2$

Assume 4-bit representation.

$$2 = 0010, \quad 1\text{'s complement of } 2 = 1101$$

$$\begin{array}{rcccc} & 0 & 1 & 1 & 0 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 1 \end{array}$$

End-around carry:

$$0011 + 1 = 0100 = +4$$

One's complement subtraction: $3 - 5$

Assume 4-bit representation.

$$5 = 0101, \quad 1\text{'s complement of } 5 = 1010$$

$$\begin{array}{r} 0 \ 0 \ 1 \ 1 \\ + \ 1 \ 0 \ 1 \ 0 \\ \hline 1 \ 1 \ 0 \ 1 \end{array}$$

No carry is produced, so the answer is negative.

$$1101 \text{ is } 1\text{'s complement of } 0010 \Rightarrow -2$$

Subtraction using two's complement

To compute $A - B$:

- 1 Compute two's complement of B : call it B_2 .
- 2 Add $A + B_2$.
- 3 If a carry is produced, ignore it.
- 4 If no carry is produced, result is negative in two's complement form.

Why this matters

Subtraction can use the same hardware as addition.

Two's complement subtraction: $6 - 2$

Assume 4-bit representation.

$$2 = 0010, \quad 2\text{'s complement of } 2 = 1101 + 1 = 1110$$

$$\begin{array}{rcccc} & 0 & 1 & 1 & 0 \\ + & 1 & 1 & 1 & 0 \\ \hline 1 & 0 & 1 & 0 & 0 \end{array}$$

Ignore carry:

$$0100 = +4$$

Two's complement subtraction: $3 - 5$

Assume 4-bit representation.

$$5 = 0101, \quad 2\text{'s complement of } 5 = 1010 + 1 = 1011$$

$$\begin{array}{r} 0 \ 0 \ 1 \ 1 \\ + \ 1 \ 0 \ 1 \ 1 \\ \hline 1 \ 1 \ 1 \ 0 \end{array}$$

No carry is produced. 1110 is the two's complement representation of -2 .

Overflow in signed addition

- Adding numbers with different signs cannot cause signed overflow.
- Overflow can occur only when both operands have the same sign.
- Overflow occurs when the sign of the sum differs from the sign of the operands.

4-bit example

$$0111(+7) + 0001(+1) = 1000$$

The result bit pattern represents -8 in 4-bit two's complement, so overflow occurred.

Overflow detection rule

For two's complement addition:

$$\text{Overflow} = C_{in \text{ to sign bit}} C_{out \text{ from sign bit}}$$

Equivalently:

- positive + positive gives negative $\Rightarrow$ overflow
- negative + negative gives positive $\Rightarrow$ overflow
- otherwise no signed overflow

Binary Coded Decimal (BCD)

- Sometimes it is useful to manipulate numbers digit-by-digit in decimal form.
- In BCD, each decimal digit is represented by its 4-bit binary equivalent.
- Six 4-bit combinations are invalid: 1010, 1011, 1100, 1101, 1110, 1111.

Examples

$23810 \rightarrow 0010\ 0011\ 1000_{BCD}$

$12.3910 \rightarrow 0001\ 0010.0011\ 1001_{BCD}$

BCD table

Decimal	BCD	Decimal	BCD
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Addition of BCD numbers

When adding BCD numbers, a correction step may be required.

Correction rule

Add $6 = (0110)_2$ to a nibble if:

- the nibble is an invalid BCD code, or
- there is a carry out of that nibble.

Idea

Binary sum of decimal digits may exceed 9; adding 6 skips the six invalid 4-bit combinations.

BCD addition example: $23 + 48$

$$23 = 0010\ 0011, \quad 48 = 0100\ 1000$$

$$\begin{array}{r} 0010 0011 \\ + 0100 1000 \\ \hline 0110 1011 \end{array}$$

Lower nibble 1011 is invalid, so add 0110:

$$1011 + 0110 = 1\ 0001$$

Final BCD:

$$0111\ 0001 = 71$$

Gray code

- Gray code is a non-weighted binary code.
- Successive code words differ in only one bit.
- This property reduces error in systems where values change gradually, e.g., shaft encoders.

Cyclic code property

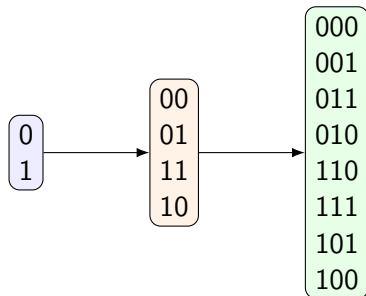
Adjacent code words differ by one bit, including the transition from last to first in a cyclic sequence.

4-bit binary and Gray code

Decimal	Binary	Gray	Decimal	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Self-reflecting construction of Gray code

- 1 Start with 1-bit Gray code: 0, 1.
- 2 Reflect the list.
- 3 Prefix 0 to the first half and 1 to the reflected half.
- 4 Repeat to generate more bits.



Binary to Gray conversion

Let binary be $b_{n-1}b_{n-2} \dots b_1b_0$ and Gray code be $g_{n-1}g_{n-2} \dots g_1g_0$.

$$g_{n-1} = b_{n-1}$$

$$g_i = b_{i+1} \oplus b_i, \quad 0 \leq i \leq n-2$$

Example

$$1011_2 \rightarrow g_3 = 1, \quad g_2 = 1 \oplus 0 = 1, \quad g_1 = 0 \oplus 1 = 1, \quad g_0 = 1 \oplus 1 = 0$$

$$1011 \rightarrow 1110_{\text{Gray}}$$

Gray to binary conversion

$$b_{n-1} = g_{n-1}$$

$$b_i = b_{i+1} \oplus g_i, \quad 0 \leq i \leq n-2$$

Example

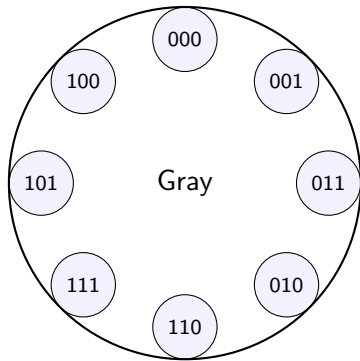
$$1110_{Gray} \rightarrow b_3 = 1$$

$$b_2 = 1 \oplus 1 = 0, \quad b_1 = 0 \oplus 1 = 1, \quad b_0 = 1 \oplus 0 = 1$$

$$1110_{Gray} = 1011_2$$

Gray code for angular rotation measurement

- In a rotary encoder, sensor readings may occur during transition between two positions.
- If ordinary binary is used, many bits may change at once.
- Gray code ensures only one bit changes between adjacent positions.



Why Gray code reduces readout error

Consider transition from decimal 7 to decimal 8:

- Binary: $0111 \rightarrow 1000$; all four bits change.
- Gray code: $0100 \rightarrow 1100$; only one bit changes.

Result

During transition, Gray code can cause at most a one-step position ambiguity, whereas binary can cause a large wrong reading.

From Binary Numbers to Binary Logic

We Now Know How Computers Represent Data

Binary representation uses only two symbols:

0 and **1**

Binary Representation

0 and 1 represent information.

$$(1011)_2 = (11)_{10}$$

Numbers, characters and other data can ultimately be represented using bits.

Binary Logic

0 and 1 can also represent two logical states.

0	1
LOW	HIGH
FALSE	TRUE
OFF	ON

Logic Gates: Building Blocks of Digital Circuits

Logic Gate

A **logic gate** takes one or more binary inputs and produces a binary output according to a logical operation.



Basic Gates

NOT
AND
OR

Universal Gates

NAND
NOR

Exclusive Gates

XOR
XNOR

Key Idea

Complex digital circuits are constructed by combining these simple logic gates.

The Three Basic Logic Operations

NOT

$$Y = \bar{A}$$

A	Y
0	1
1	0

Inverts the input

AND

$$Y = A \cdot B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Output is 1 only when **both inputs are 1.**

OR

$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Output is 1 when **at least one input is 1.**

Truth Table

A truth table lists **all possible input combinations** and the corresponding output of a logic function.

Boolean Algebra: Basic Concepts

Boolean Algebra

An algebraic system over the set $\{0, 1\}$ with three basic operations.

Operation	Name	Notation
AND	Logical product	$x \cdot y$ or $x \wedge y$
OR	Logical sum	$x + y$ or $x \vee y$
NOT	Complement	x' or $\bar{x}$

Boolean Variables and Expressions

A **Boolean variable** takes only 0 or 1. A **Boolean expression** is formed using variables, constants and Boolean operators.

Laws of Boolean Algebra

1. Basic Identities

$$x + 0 = x, \quad x + 1 = 1$$

$$x \cdot 1 = x, \quad x \cdot 0 = 0$$

2. Idempotent Laws

$$x + x = x \quad x \cdot x = x$$

3. Complementation Laws

$$x + x' = 1, \quad x \cdot x' = 0$$

$$(x')' = x$$

4. Commutative Laws

$$x + y = y + x$$

$$xy = yx$$

5. Associative Laws

$$(x + y) + z = x + (y + z)$$

$$(xy)z = x(yz)$$

6. Distributive Laws

$$x(y + z) = xy + xz$$

$$x + yz = (x + y)(x + z)$$

7. Absorption Laws

$$x + xy = x$$

$$x(x + y) = x$$

8. Useful Laws

$$x + x'y = x + y$$

$$x(x' + y) = xy$$

9. Consensus Theorem

$$xy + x'z + yz = xy + x'z$$

$$(x + y)(x' + z)(y + z) = (x + y)(x' + z)$$

De Morgan's Theorems

Two Variables

$$(x + y)' = x'y'$$

$$(xy)' = x' + y'$$

Meaning

When a complement crosses a bracket, AND and OR interchange.

Multiple Variables

$$(x_1 + x_2 + \cdots + x_n)' = x_1'x_2'\cdots x_n'$$

$$(x_1x_2\cdots x_n)' = x_1' + x_2' + \cdots + x_n'$$

Rule

Complement every variable and interchange + with $\cdot$.

Literals, Minterms and Maxterms

Literal

A **literal** is a Boolean variable or its complement, e.g., x or x' .

Minterm

An AND term containing **every variable exactly once**, either complemented or uncomplemented.

Product of literals

Maxterm

An OR term containing **every variable exactly once**, either complemented or uncomplemented.

Sum of literals

Remember

Minterm = AND of all variables

Maxterm = OR of all variables

Properties of Minterms and Maxterms

Minterms

For n variables, there are 2^n minterms.

Each minterm is 1 for exactly one input row.

Maxterms

For n variables, there are 2^n maxterms.

Each maxterm is 0 for exactly one input row.

Minterm
identifies a 1-row



Maxterm
identifies a 0-row

Canonical Representation of Boolean Functions

Canonical Form

A standard representation in which every term contains all input variables.

Canonical SOP

Sum of Products

Sum of minterms

Also called disjunctive normal form.

Canonical POS

Product of Sums

Product of maxterms

Also called conjunctive normal form.

Truth Table Connection

Canonical SOP uses output-1 rows; canonical POS uses output-0 rows.

Canonical SOP and POS from a Truth Table

Canonical SOP

Use rows where $F = 1$.

- 1 Select each output-1 row.
- 2 Form the corresponding minterm.
- 3 Input 0 $\rightarrow$ complemented variable.
- 4 Input 1 $\rightarrow$ uncomplemented variable.
- 5 OR all selected minterms.

$$F = \sum m(\dots)$$

Canonical POS

Use rows where $F = 0$.

- 1 Select each output-0 row.
- 2 Form the corresponding maxterm.
- 3 Input 0 $\rightarrow$ uncomplemented variable.
- 4 Input 1 $\rightarrow$ complemented variable.
- 5 AND all selected maxterms.

$$F = \prod M(\dots)$$

Rule

SOP $\rightarrow$ 1s $\rightarrow$ minterms

POS $\rightarrow$ 0s $\rightarrow$ maxterms

Converting Boolean Expressions to Canonical Form

Canonical SOP

If a product term is missing variable x , multiply by 1:

$$x + x' = 1$$

$$P = P(x + x')$$

Expand and repeat until every product term contains all variables.

Canonical POS

If a sum term is missing variable x , add 0:

$$xx' = 0$$

$$S = S + xx'$$

Use

$$A + BC = (A + B)(A + C)$$

until every sum term contains all variables.

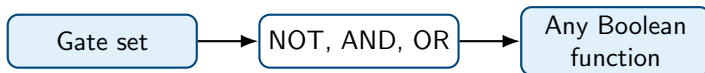
Canonical Requirement

Every minterm or maxterm must contain every input variable exactly once.

Functional Completeness

Definition

A gate set is **functionally complete** if every Boolean function can be implemented using only gates from that set.



NAND

Universal gate

NOR

Universal gate

NAND as a Universal Gate

NAND Operation

$$A \uparrow B = (AB)'$$

Function	Using NAND	Meaning
NOT	$A \uparrow A = (AA)' = A'$	Tie both inputs together.
AND	$[(AB)']' = AB$	NAND once, then complement using NAND.
OR	$A + B = (A'B')'$	Generate A' and B' using NAND, then NAND them.

Conclusion

Since NOT, AND and OR can be built using NAND, NAND is functionally complete.

NOR as a Universal Gate

NOR Operation

$$A \downarrow B = (A + B)'$$

Function	Using NOR	Meaning
NOT	$A \downarrow A = (A + A)' = A'$	Tie both inputs together.
OR	$[(A + B)']' = A + B$	NOR once, then complement using NOR.
AND	$AB = (A' + B')'$	Generate A' and B' using NOR, then NOR them.

Conclusion

Since NOT, OR and AND can be built using NOR, NOR is functionally complete.

Karnaugh Map (K-Map): Boolean Simplification

Why K-Map?

A **Karnaugh Map** is a graphical method for simplifying Boolean functions without lengthy algebraic manipulation.

Basic Idea

For an n -variable function:

$$\text{Number of cells} = 2^n$$

Each cell represents one **minterm**.

Adjacent cells differ in exactly **one variable**.

Grouping Rules

Group adjacent 1s in powers of two:

$$1, 2, 4, 8, 16, \dots$$

- Make groups as **large as possible**.
- Every 1 must be covered.
- Groups may overlap.
- Edge cells can be adjacent.

3-Variable and 4-Variable K-Maps

3-Variable K-Map

$A \setminus BC$	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

$2^3 = 8$ cells

4-Variable K-Map

$AB \setminus CD$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

$2^4 = 16$ cells

Important: Gray-Code Ordering

Rows/columns follow

$00 \rightarrow 01 \rightarrow 11 \rightarrow 10$

not binary order 00, 01, 10, 11.

Thus adjacent cells differ in only **one bit**. The left/right edges and top/bottom edges are also adjacent.

K-Map Minimization: SOP, POS and Don't Cares

Minimum SOP

Group the 1s

- Place minterms in K-map.
- Group adjacent 1s.
- Use largest possible groups.
- Each group gives a **product term**.

$$F = P_1 + P_2 + \dots$$

Minimum POS

Group the 0s

- Place 0s using maxterms.
- Group adjacent 0s.
- Use largest possible groups.
- Each group gives a **sum term**.

$$F = S_1 S_2 \dots$$

Don't Care: X

Some input combinations may never occur, so their outputs do not matter.

$$X = 0 \text{ or } 1$$

Use an X only when it helps create a **larger group**.

- X may be included.
- X may be ignored.
- X need not be covered.

Golden Rule

SOP $\Rightarrow$ group 1s

POS $\Rightarrow$ group 0s

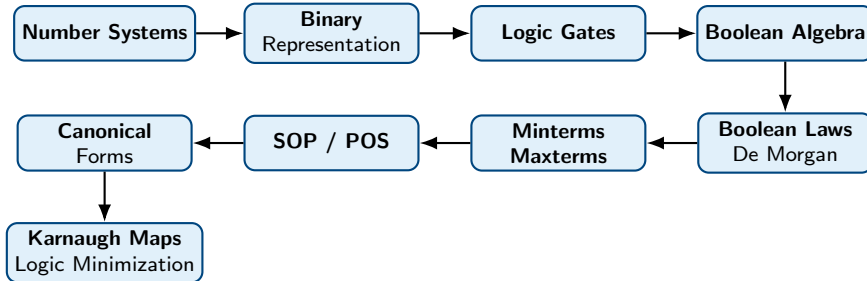
Always make the groups as large as possible.

From Problem to Digital Logic

Where We Started

Problem → Algorithm → Software → ISA → Microarchitecture → Digital Logic → Devices → Electrons

Zooming into Digital Logic



Textbook

M. Morris Mano and Michael D. Ciletti,
Digital Design: With an Introduction to the Verilog HDL, Pearson.

- **Chapter 1:** Digital Systems and Binary Numbers
- **Chapter 2:** Boolean Algebra and Logic Gates
- **Chapter 3:** Gate-Level Minimization

Online Lecture Material

- **Prof. Indranil Sengupta**, IIT Kharagpur
Switching Circuits and Logic Design, NPTEL.
- **Prof. Onur Mutlu**, ETH Zürich
Lecture material referred to for the introductory **levels-of-abstraction / hierarchy diagram**.

Acknowledgements

Teaching Material

- Lecture concepts, examples, and illustrations were adapted/referred from the publicly available teaching material of **Prof. Indranil Sengupta**, IIT Kharagpur (NPTEL).
- The introductory computer-system abstraction / hierarchy diagram was adapted from lecture material by **Prof. Onur Mutlu**, ETH Zürich.

AI-Assisted Preparation

- **ChatGPT (OpenAI)** was used as an AI-assisted tool for organizing, formatting, and preparing portions of these lecture slides.

The above resources are gratefully acknowledged for educational and teaching purposes.